

Tham số	Giá trị hiện tại	Cách đo
<code>robot_width</code>	0.40m	Đo trực tiếp bề ngang xe (đã xác nhận)
<code>margin_a2</code>	0.05m	Đo khoảng hờ an toàn mong muốn 2 bên xe, hoặc để tạm rồi tăng/giảm sau khi thấy va chạm/né hụt thực tế

Nhóm 2 — Ngưỡng chuyển mode (`should_enter/exit_avoidance`)

Tham số	Giá trị hiện tại	Cách đo/tính chỉnh
<code>enter_dist</code>	0.40m	Đặt vật cản ở nhiều khoảng cách, quan sát xe có phản ứng đủ sớm để né an toàn không (tính cả quãng đường phanh/xoay ở tốc độ tối đa). Nếu xe né trễ → tăng lên
<code>exit_dist</code>	0.60m	Phải luôn lớn hơn <code>enter_dist</code> (hysteresis). Test xem xe có bị kẹt lâu trong mode né dù vật cản đã qua không → nếu kẹt quá lâu, giảm nhẹ; nếu thoát quá sớm rồi né lại ngay → tăng
<code>straight_ang_th</code>	10°	Cho xe chạy bám line ở đoạn cua thật, log giá trị <code>ang</code> thực tế kinect trả về khi xe "coi như đang rẽ" theo cảm quan, đổi chiều ngưỡng
<code>ang_hysteresis</code>	5°	Quan sát log <code>ang_state</code> có nhảy qua lại liên tục ở đoạn cua nhẹ không — nếu có, tăng giá trị này
<code>err_exit_th</code>	0.05m	Quan sát giá trị <code>error</code> thực tế khi xe bám line ổn định (không dao động) — đặt ngưỡng gần bằng mức dao động tự nhiên đó
<code>debounce_frames</code>	4 (≈200ms @ 20Hz)	Cho xe chạy qua môi trường có nhiều lidar thực tế, đếm xem cần bao nhiêu frame liên tiếp mới lọc hết nhiễu giả (chuyển mode sai)
<code>data_timeout</code>	0.5s	Kiểm tra tần số publish thực tế của <code>/scan</code> và <code>/kinect/line_data</code> (dùng <code>rostopic Hz</code>), đặt <code>data_timeout</code> lớn hơn khoảng vài lần chu kỳ publish bình thường để không bị false-trigger

Nhóm 3 — Thuật toán trường thể (`compute_avoidance_cmd`) — nhóm quan trọng và khó đoán nhất

Tham số	Giá trị hiện tại	Cách đo/tinh chỉnh
<code>g_obs</code>	0.50m	Ngưỡng phát hiện dải vật cản trong thuật toán trường thể — nên thử đặt bằng <code>enter_dist</code> trước để đồng nhất, sau đó tinh chỉnh riêng nếu cần
<code>g_star</code>	0.60m	Ngưỡng kích hoạt lực đẩy — phải test bằng cách đặt vật cản gần dần, quan sát tại khoảng cách nào xe cần "né gấp" hơn
<code>k_A</code>	1.0	Hệ số lực hút về điểm đích tạm — không có đơn vị vật lý chuẩn , phải dò bằng thực nghiệm: tăng nếu xe "lười" bám sát điểm né, giảm nếu xe rẽ quá gắt
<code>k_R</code>	0.03	Hệ số lực đẩy — nhạy nhất trong toàn bộ hệ thống vì công thức có <code>1/g³</code> , chỉ cần sai 1 chút cũng có thể khiến xe phản ứng cực đoan khi vật cản rất gần. Nên bắt đầu từ giá trị rất nhỏ, tăng dần từng bước nhỏ, test kỹ ở khoảng cách gần nguy hiểm
<code>d_off</code>	0.15m	Độ lệch điểm đích khỏi mép vật cản — liên quan trực tiếp đến khoảng cách an toàn xe giữ khi lướt qua vật cản, nên đo bằng cách quan sát trực tiếp khoảng hở thực tế giữa xe và vật cản lúc né
<code>side_lock_frames</code>	3 (≈150ms)	Cân bằng giữa độ trễ chốt mép và độ tin cậy — quan sát xem 3 frame có đủ để tránh chốt sai mép do nhiễu 1 frame không

Nhóm 4 — Điều khiển tốc độ và góc quay (`v`, `w`)

Tham số	Giá trị hiện tại	Cách đo/tinh chỉnh
<code>v_max_avoid</code>	0.4 m/s	Đã chốt theo bạn — nhưng nên verify lại bằng cách đo thực tế tốc độ tối đa an toàn của xe trên bề mặt track thật (ma sát, độ trơn)
<code>v_min_avoid</code>	0.1 m/s	Đã chốt theo bạn — kiểm tra xe có thực sự bò được ở tốc độ này không (motor có bị "chết máy" ở tốc độ quá thấp không)

Tham số	Giá trị hiện tại	Cách đo/tính chính
<code>safety_min_dist</code>	0.15m	Cần đo: khoảng cách tối thiểu xe có thể dừng lại an toàn kể từ lúc phát hiện vật cản ở <code>v_max</code> , tính luôn độ trễ hệ thống (lidar → xử lý → serial → động cơ phản ứng)
<code>k_theta</code>	1.5	Hệ số P chuyển góc lệch → vận tốc góc — tăng nếu xe quay chậm/lê mê khi né, giảm nếu xe quay giật/quá nhạy
<code>w_max</code>	1.5 rad/s ($\approx 86^\circ/\text{s}$)	Giới hạn cơ khí thực tế của xe — cần tra thông số động cơ/bánh xe hoặc đo thực nghiệm tốc độ quay tối đa ổn định
<code>w_smooth_alpha</code>	0.3	Tăng nếu muốn xe phản ứng nhanh hơn (chấp nhận giật nhẹ), giảm nếu cần êm hơn (chấp nhận trễ)

Nhóm 5 — Độ trễ hệ thống (chưa đo, nhưng ảnh hưởng đến toàn bộ các ngưỡng trên)

Đây là phần **quan trọng nhưng dễ bị bỏ qua**: mọi ngưỡng khoảng cách (`enter_dist`, `g_star`, `safety_min_dist`) đều phải tính thêm **quãng đường xe đi được trong thời gian trễ** từ lúc lidar đo → tính toán → gửi serial → Arduino phản ứng → động cơ thực sự đổi tốc độ/góc. Nếu độ trễ này lớn (ví dụ 100-200ms) mà xe đang chạy `v_max = 0.4 m/s`, xe đã đi thêm 4-8cm trước khi lệnh né có hiệu lực — cần cộng dự phòng này vào các ngưỡng an toàn.

Cách đo: dùng `rostopic Hz /scan` và `rostopic Hz /kinect/line_data` để biết tần số cập nhật, đo thời gian xử lý trong `control_loop` (thêm log timestamp), và đo độ trễ Serial thực tế (gửi lệnh, quan sát bằng mắt/camera tốc độ cao khi nào động cơ phản ứng).

Gợi ý thứ tự test thực nghiệm

- Nhóm 5 trước tiên** — đo độ trễ hệ thống, vì nó ảnh hưởng tới cách chọn mọi ngưỡng khác
- Nhóm 1** — đã có sẵn, xác nhận lại
- Nhóm 2** — test chuyển mode bằng vật cản tĩnh, môi trường trơn tru như bạn mô tả
- Nhóm 4** (`v_min/max`, `safety_min_dist`, `w_max`) — test giới hạn vật lý xe trước khi đụng tới thuật toán trường thế
- Nhóm 3** (`k_A`, `k_R`, `d_off`, `g_star`) — phần khó nhất, nên test cuối cùng, bắt đầu từ tốc độ chậm, tăng dần độ khó (vật cản gần hơn, xe nhanh hơn)